

Kotonoha 構想

意味履歴を管理する Semantic Lineage System

Kotonoha とは、Git などの内容履歴を物理的アンカーとして用いながら、文書・議論・設計・実装における意味状態と意味変化 ΔM を記録し、RDE によって監査可能にする Semantic Lineage System である。

作成日 2026-05-17T00:00:00+09:00

更新日 2026-05-17T00:00:00+09:00

Author Tomoyuki Kano <tomyuk@zyxcorp.jp>

Status draft

Document Type concept_note

目次

1	中心命題	3
2	背景：Git だけでは意味の履歴を扱えない	3
3	基本定義	3
4	基本データモデル	4
5	Document Object との相互作用	5
6	Git との関係	5
7	Kotonoha Core	6
8	最小 Core の DB 方針	6
9	RDE との関係	7
10	クライアント群の位置づけ	7
10.1	ChatGPT app	8
10.2	Cursor / VSCode	8
10.3	Claude Code	8
10.4	NotebookLM	8
10.5	Obsidian	9
11	MCP 接続	9
12	Kotonoha Minimal UI	9
13	車輪の再発明を避ける設計原則	10
14	開発方針	11
15	Phase 0：概念仕様の固定	11
15.1	目的	11
15.2	主な成果物	12
15.3	完了条件	12
16	Phase 1：Kotonoha Core 最小実装	12
16.1	目的	12
16.2	主な実装対象	12
16.3	優先機能	13
16.4	完了条件	13
17	Phase 2：Kotonoha Minimal UI	13

17.1 目的	13
17.2 位置づけ	13
17.3 主な機能	14
17.4 完了条件	14
18 Phase 3：GitHub / Issue / PR 連携	14
18.1 目的	14
18.2 主な機能	15
18.3 完了条件	15
19 Phase 4：外部エージェント連携	15
19.1 目的	15
19.2 主要連携	16
19.3 必要となる構成	16
19.4 完了条件	16
20 Phase 5：チーム利用・制度化	16
20.1 目的	17
20.2 主な機能	17
20.3 完了条件	17
21 Phase X：将来研究	17
22 マイルストーン一覧	17
23 v0.1 の明確な非対象	18
24 Multimodal MeaningDelta の位置づけ	18
25 RDE 差異検証メモ	19
25.1 保存された要素	19
25.2 変換された要素	20
25.3 補完された要素	20
25.4 未解決の要素	20
25.5 逸脱リスク	20
25.6 次回更新方針	20
26 要約	20
付録Obsidian 用メタデータ	21

1 中心命題

Kotonoha は、文書・議論・設計・実装の変更履歴ではなく、その背後にある**意味変化 ΔM** と知的関係の系譜を記録・評価・再利用するための Semantic Lineage System である。

Git が「文字列とファイルの履歴」を扱うなら、Kotonoha は「意味状態と意味変化の履歴」を扱う。したがって、Kotonoha の本質は、特定のエディタ、ノートアプリ、AI コーディングツールではない。それらを横断して、何が保存され、何が変換され、何が補完され、何が逸脱したのかを記録する基盤である。

最短定義

Kotonoha とは、Git などの内容履歴を物理的アンカーとして用いながら、文書・議論・設計・実装における意味状態と意味変化 ΔM を記録し、RDE によって監査可能にする Semantic Lineage System である。

2 背景：Git だけでは意味の履歴を扱えない

現代の知的作業では、Git、Obsidian、Cursor、ChatGPT app、Claude Code、NotebookLM などが組み合わされる。

それぞれは強力である。Git は差分を保存する。Obsidian は知識のリンクを扱う。Cursor / VSCode は実装作業を支援する。ChatGPT app は構想・批評・仕様化を支援する。Claude Code はコードベース単位の実装作業を担える。NotebookLM は資料読解・要約・構造化に強い。

しかし、これらを組み合わせても、なお欠けているものがある。それは、**意味がどのように変わったかの履歴**である。

たとえば、文章が書き換えられたとき、Git は行差分を示せる。しかし、次のような問いには直接答えられない。

- この変更で主張の射程は広がったのか
- 仮説が断定に変わっていないか
- 元の思想は保存されたのか
- 未解決点が消されていないか
- AI による補完は正当な発展なのか、逸脱なのか
- 実装上の都合が理論的主張へすり替わっていないか

Kotonoha は、この欠落を埋めるための基盤である。

3 基本定義

Kotonoha は、以下の主要オブジェクトを扱う。

MeaningState

ある時点における意味状態。

MeaningDelta

意味状態間の変化 ΔM 。

RDEAssessment

意味変化に対する RDE 評価。

SourceContext

参照元・議論・資料・会話・Issue。

ClaimGraph

主張・概念・根拠・依存関係。

UnresolvedGap

未解決点・判断保留・追加検証事項。

ReviewDecision

人間または制度的判断。

AgentRun

AI エージェントによる作業単位。

Kotonoha が扱うのは、単なるファイルではなく、**意味を帯びた作業単位**である。

4 基本データモデル

Kotonoha の中心線は次の通りである。

```
SourceContext
→ MeaningState
→ MeaningDelta /  $\Delta M$ 
→ RDEAssessment
→ ReviewDecision
→ New MeaningState
```

この構造により、ある資料・会話・Issue・Git 差分から意味状態が抽出され、その意味状態が改稿・生成・実装・レビューを通じて変化し、その変化が RDE により評価され、人間または制度的判断によって採用・保留・差し戻される。

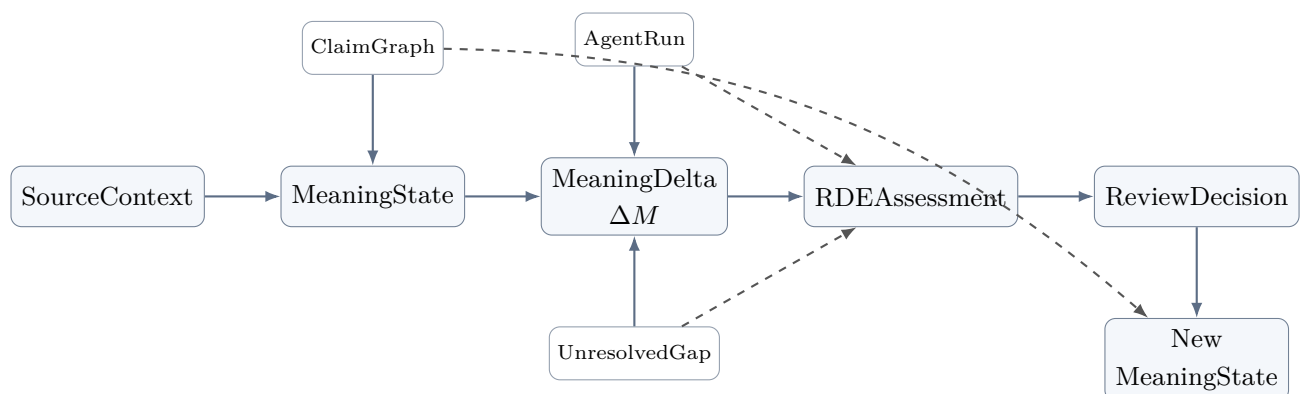


図1 Kotonoha の基本データモデル

この図は、Kotonoha が単なる保存庫ではなく、**意味状態が変化し、その変化が評価され、再び意味状態として定着する循環システム**であることを示している。

5 Document Object との相互作用

Kotonoha では、文書を単なるファイルではなく、知的作業の対象としての **Document Object** として扱う。

Document Object には、論文草稿、仕様書、note 記事、設計メモ、Issue 下書き、実験記録などが含まれる。

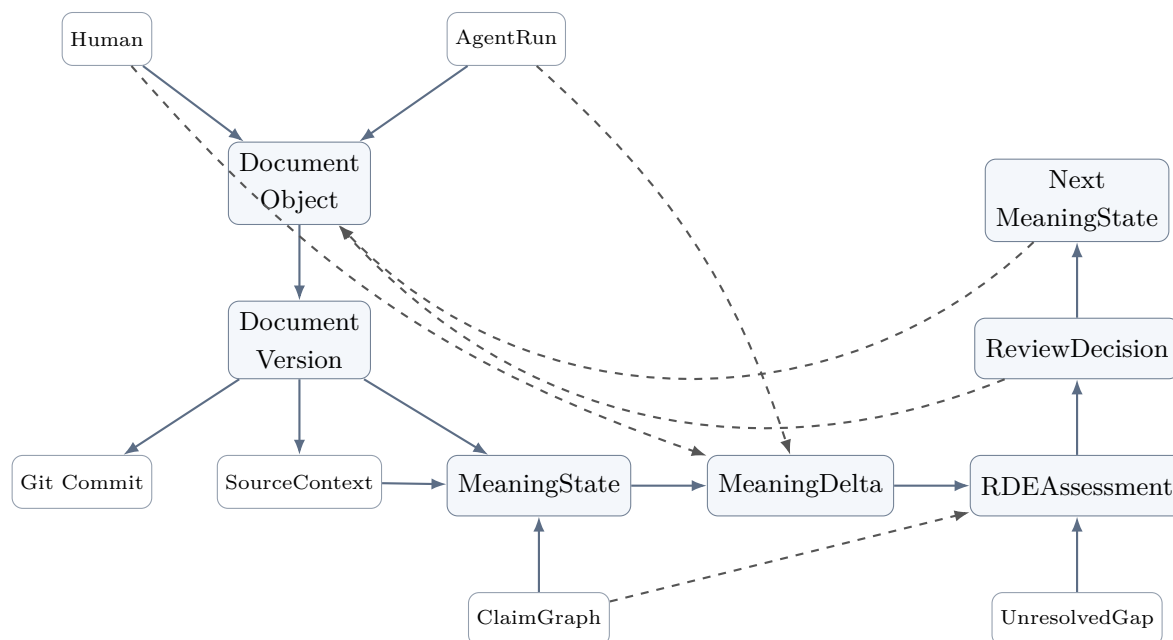


図 2 Document Object を中心とする相互作用

この図の要点は、文書の物理的変更と意味的変更を分けることである。

Document Object



Document Version / Git Commit

= 物理的・文字列的な履歴

MeaningState / MeaningDelta / RDEAssessment

= 意味的・評価的な履歴

Git は文書の版を固定し、Kotonoha はその版が何を意味していたかを記録する。

6 Git との関係

Kotonoha は Git を置き換えない。むしろ、Git を物理的・文字列的な履歴のアンカーとして使う。

```
Git
= content lineage
= ファイル・文字列・commit・branch・tag・merge の履歴

Kotonoha DB
= semantic lineage
= 意味状態・意味変化・評価・未解決点・判断履歴の履歴
```

Git の commit hash は、Kotonoha DB にとって固定点になる。

```
Git commit / diff / file path / line range
      ↓
Kotonoha MeaningDelta / RDEAssessment / ReviewDecision
```

一言で言えば、Git は文字列の時間軸を持ち、Kotonoha は意味の時間軸を持つ。

7 Kotonoha Core

Kotonoha の本体は、特定の UI ではなく Core である。

```
Kotonoha Core
├── Semantic Lineage DB
├── Git / Content Adapter
├── MeaningDelta Engine
├── RDE Engine
├── Claim / Source Manager
├── AgentRun Logger
├── Workflow / Review Engine
└── MCP Gateway
```

Core は、エディタ非依存、クライアント非依存である。

Obsidian、Cursor、VSCode、ChatGPT app、Claude Code、NotebookLM は、すべて Kotonoha Core に接続される外部クライアントまたはエージェントである。

8 最小 Core の DB 方針

Kotonoha の最小 Core における DB は、PostgreSQL を前提とする。

SQLite はローカル単体の試作には便利だが、Kotonoha では最初から以下が重要になる。

- 複数文書・複数 commit 間の意味履歴
- JSONB による柔軟な RDE 記録
- full-text search
- pgvector 等による将来の意味検索
- transaction による監査記録の一貫性
- GitHub / VSCode / ChatGPT app / Claude Code 等からの接続
- 将来のチーム利用

したがって、正式な最小 Core は次のように置く。

```
Kotonoha Core v0.1
├─ PostgreSQL
├─ DB migration
├─ Git Adapter
├─ Document Object Registry
├─ MeaningState schema
├─ MeaningDelta schema
├─ RDEAssessment schema
├─ ReviewDecision schema
├─ AgentRun schema 最小版
└─ CLI
```

PostgreSQL は Phase 1 から採用し、将来的にチーム運用・Web Console・権限管理へ拡張する。

9 RDE との関係

Kotonoha は RDE を内包する。

RDE (Resonant Deviation Evaluator) は、生成物や改稿結果が元の意味状態からどう変化したかを評価する層である。

RDE は、単なる品質評価器ではない。評価対象は、次のような差異である。

preserved

保存された要素。

authorized transformation

許可された変換。

inferred extension

補完・推論された拡張。

unresolved gap

未解決のまま残した要素。

suspicious drift

疑わしい逸脱。

critical distortion

重大な歪曲。

Kotonoha は、RDE が評価するための対象、履歴、文脈、再審可能性を保持する土台である。

言い換えると、RDE が意味変化を裁定するなら、Kotonoha は意味変化を記憶する。

10 クライアント群の位置づけ

Kotonoha は、特定のアプリケーションに閉じない。

10.1 ChatGPT app

ChatGPT app は、構想整理、論文・設計思想、RDE 差異検証、仕様生成に向く。

```
ChatGPT app
→ Meaning Design
→ RDE Discussion
→ Spec Draft
→ Kotonoha DB へ記録
```

ChatGPT app は、意味設計・意味評価ワークベンチとして位置づけられる。

10.2 Cursor / VSCode

Cursor / VSCode は、実装、文書編集、リポジトリ内作業、Git diff との接続に向く。

```
Cursor / VSCode
→ File Edit
→ Git Diff
→ MeaningDelta Registration
→ RDE Panel
```

Cursor は必須要件ではなく、VSCode 系クライアントの一つである。

10.3 Claude Code

Claude Code は、コードベース読解、実装修正、テスト修正、PR 単位の作業代行に向く。

```
Kotonoha Spec
→ Claude Code AgentRun
→ Patch / Test Result
→ Git Diff
→ RDE Assessment
→ Human Review
```

Claude Code は、Kotonoha の実装系エージェントとして有望である。ただし、Kotonoha Core の意味履歴と RDE 監査に従属する作業実行器として扱うべきである。

10.4 NotebookLM

NotebookLM は、資料読解、要約、FAQ、Mind Map、Audio Overview などに向く。

```
Source Set
→ NotebookLM Interpretation
→ Derived Artifact
→ MeaningDelta
→ RDE Assessment
```

NotebookLM の出力は正解ではなく、Kotonoha 上では意味変換 ΔM の候補として扱われる。

10.5 Obsidian

Obsidian は、知識ノート、草稿、リンク構造、個人研究環境に向く。

ただし、Kotonoha は Obsidian plugin に閉じない。Obsidian は Kotonoha Console の一候補である。

11 MCP 接続

Kotonoha は MCP 接続によって、外部エージェント・外部知識処理器と接続できる。

ただし、MCP は「何でも自由につなぐ線」ではなく、capability 制御された接続層として扱う。

```
Kotonoha MCP Gateway
├─ read_source_set
├─ export_context_pack
├─ register_derived_artifact
├─ create_meaning_delta
├─ run_rde_assessment
├─ attach_human_review
└─ export_report
```

NotebookLM、ChatGPT app、Claude Code、Cursor などは、MCP または専用 adapter を通じて Kotonoha Core に接続される。

重要なのは、外部ツールの出力を正解と見なさないことである。それらはすべて、Kotonoha 上では意味変換 ΔM の候補として扱われ、RDE と人間レビューの対象になる。

12 Kotonoha Minimal UI

VSCode plugin は、Obsidian/Cursor の代替ではなく、Kotonoha の最小基本 UI になり得る。

最小 UI の目的は、編集そのものではない。

Diff を見る UI ではない。
Diff に意味を与える UI である。

または、次のように言える。

編集するための UI ではない。
編集が何を変えたのかを記録する UI である。

Kotonoha Minimal UI v0.1 は、次を扱う。

```

Kotonoha Minimal UI
├── Current Context
│   ├── repo
│   ├── branch
│   ├── file
│   ├── selection
│   └── git diff
├── MeaningDelta Form
│   ├── intended change
│   ├── preserved elements
│   ├── transformed elements
│   ├── inferred extensions
│   ├── unresolved gaps
│   └── drift risks
├── RDE Panel
│   ├── assessment result
│   ├── warning
│   ├── required human review
│   └── next action
└── Register / Link
    ├── save to Kotonoha DB
    ├── link to commit
    ├── link to issue / PR
    └── export report

```

この UI は、VSCode extension として先行実装する価値がある。理由は、VSCode / Cursor 圏では Git diff、ファイル、選択範囲、branch、commit、テスト、Issue 草稿が近くにあるためである。

13 車輪の再発明を避ける設計原則

Kotonoha は、既存ツールを再実装しない。
再実装すべきでないものは次の通りである。

- Obsidian 的なノートアプリ全体
- Cursor 的な AI コードエディタ
- NotebookLM 的な資料読解 UI
- GitHub 的な Issue / PR 管理
- Git そのもの
- 汎用 LLM チャット UI

Kotonoha が作るべきものは次の通りである。

- 意味状態の記録
- 意味変化 ΔM の登録
- Git diff と意味差分の対応付け
- RDE 評価の保存
- 未解決点の保持
- 人間レビューと判断履歴
- 外部エージェント出力の意味監査

つまり、既存ツールが作業を行い、Kotonoha が意味の履歴を記録する。

14 開発方針

Kotonoha の開発は、最初から大規模な統合知識基盤を目指すのではなく、意味履歴を記録・監査する最小構造から開始する。

初期段階では、対象を主に以下に限定する。

- Markdown / LaTeX / 設計文書
- Git 管理されたファイル
- Git diff / commit
- MeaningState
- MeaningDelta
- RDEAssessment
- ReviewDecision
- 最小 UI

開発原則は次の通りである。

1. Core first

UI や外部ツール連携より先に、Kotonoha Core のデータモデルを確立する。

2. Git anchored

Git commit / diff / file path / line range を意味履歴の物理アンカーとして利用する。

3. UI minimal

最初の UI は、編集環境そのものではなく、差分に意味を与える最小 UI とする。

4. Human-in-the-loop

RDE 評価は最終判断ではなく、人間レビューと再審可能性を前提とする。

5. Tool agnostic

Obsidian、Cursor、VSCode、ChatGPT app、Claude Code、NotebookLM は交換可能なクライアントまたは外部エージェントとして扱う。

15 Phase 0：概念仕様の固定

15.1 目的

Kotonoha の中核概念を、実装可能な仕様へ落とし込む。

15.2 主な成果物

- Kotonoha 構想ドキュメント v0.1
- 用語定義
- 基本データモデル
- Document Object 図解
- Git 連携モデル
- RDE との関係整理
- v0.1 で扱わない範囲の明示

15.3 完了条件

- MeaningState / MeaningDelta / RDEAssessment / ReviewDecision の定義がある
- Document Object と Git Commit の関係が図示されている
- Kotonoha Core と外部クライアントの境界が明確である
- マルチモーダル ΔM が将来研究扱いとして分離されている

16 Phase 1 : Kotonoha Core 最小実装

16.1 目的

Git 差分に対して MeaningDelta と RDE 記録を付与できる最小バックエンドを作る。

16.2 主な実装対象

```
Kotonoha Core v0.1
├── PostgreSQL
├── DB migration
├── Git Adapter
├── Document Object Registry
├── MeaningState schema
├── MeaningDelta schema
├── RDEAssessment schema
├── ReviewDecision schema
├── AgentRun schema 最小版
└── CLI
```

16.3 優先機能

```
- kotonoha init
- kotonoha status
- kotonoha diff
- kotonoha delta create
- kotonoha rde attach
- kotonoha review approve / hold / reject
- kotonoha export
```

16.4 完了条件

```
- PostgreSQL 上に基本スキーマを作成できる
- Git 管理下の文書に対して MeaningDelta を作成できる
- MeaningDelta が commit hash / file path / line range に紐づく
- RDEAssessment を JSONB として保存できる
- ReviewDecision を記録できる
- CLI から一連の操作が可能である
```

17 Phase 2 : Kotonoha Minimal UI

17.1 目的

VSCode / Cursor 系環境で、Git 差分に意味を付与する最小 UI を提供する。

17.2 位置づけ

Kotonoha Minimal UI は、Obsidian や Cursor の代替ではない。それは、差分を意味変化として登録し、RDE 評価と人間レビューを結びつけるための最小作業面である。

17.3 主な機能

```
Kotonoha Minimal UI
├─ Current Context 表示
│   ├── repository
│   ├── branch
│   ├── file
│   ├── selection
│   └─ git diff
├─ MeaningDelta 入力
│   ├── intended change
│   ├── preserved elements
│   ├── transformed elements
│   ├── inferred extensions
│   ├── unresolved gaps
│   └─ drift risks
├─ RDE Panel
│   ├── assessment result
│   ├── warnings
│   ├── human review required
│   └─ next action
└─ Register / Link
    ├── save to Kotonoha DB
    ├── link to commit
    ├── link to issue / PR
    └─ export report
```

17.4 完了条件

- VSCode 上で現在の Git 差分を取得できる
- 選択範囲または diff 単位で MeaningDelta を登録できる
- RDEAssessment を表示できる
- ReviewDecision を UI から記録できる
- Kotonoha Core と通信できる

18 Phase 3 : GitHub / Issue / PR 連携

18.1 目的

意味履歴を、GitHub 上の Issue、Pull Request、Review Comment と接続する。

18.2 主な機能

- Issue と MeaningDelta の紐づけ
- PR diff と RDEAssessment の紐づけ
- PR 説明文への RDE サマリー出力
- ReviewDecision の GitHub comment 反映
- semantic check の CI 連携

18.3 完了条件

- PR 単位で MeaningDelta を一覧できる
- RDE 評価結果を PR レビューに添付できる
- 未解決点が Issue として再登録できる
- CI 上で最低限の semantic check が実行できる

19 Phase 4：外部エージェント連携

19.1 目的

ChatGPT app、Claude Code、NotebookLM、Obsidian などの外部ツールを、Kotonoha Core に接続する。

19.2 主要連携

ChatGPT app

- 構想整理
- 仕様生成
- RDE 差異検証
- MeaningDelta 草案生成

Claude Code

- 実装作業
- テスト修正
- patch 生成
- AgentRun 記録

NotebookLM

- Source Set 読解
- 要約
- FAQ
- Mind Map
- 派生 Artifact 登録

Obsidian

- 研究ノート
- 草稿管理
- 文書リンク
- Kotonoha Console 候補

19.3 必要となる構成

- AgentRun schema
- Context Pack export
- Derived Artifact registry
- MCP Gateway または専用 Adapter
- capability 制御
- audit log

19.4 完了条件

- 外部エージェントの出力を AgentRun として記録できる
- AgentRun から MeaningDelta を生成できる
- 外部出力を RDEAssessment 対象として扱える
- 直接 commit/push などの危険操作を制限できる

20 Phase 5：チーム利用・制度化

20.1 目的

個人研究環境から、チーム・組織で利用できる意味履歴基盤へ拡張する。

20.2 主な機能

- PostgreSQL のマルチユーザー運用
- Web Console
- Team / Organization workspace
- RBAC / capability security
- audit log hardening
- review workflow
- project-level semantic lineage
- report export

20.3 完了条件

- 複数ユーザーで MeaningDelta を共有できる
- ReviewDecision に権限管理を適用できる
- Project 単位で意味履歴を追跡できる
- 外部監査向けレポートを出力できる

21 Phase X：将来研究

以下は、Kotonoha v0.1 の中核実装には含めない。ただし、Kotonoha が意味変化一般の基盤へ拡張される可能性として、研究ノートに保持する。

- Multimodal MeaningDelta
- 画像への ΔM 評価
- 音声・音楽への ΔM 評価
- 動画編集における意味変化監査
- Temporal / Affective / Cross-modal Alignment 軸
- drift subtype の拡張
- Counterfactual replay
- 高度な ClaimGraph
- 自動 merge ではなく意味再審としての統合

これらは、実装済み能力として記述しない。あくまで、将来研究・将来実装の射程として扱う。

22 マイルストーン一覧

M0: Concept Freeze

Kotonoha 構想、用語、基本データモデル、対象範囲を固定する。

M1: Core Prototype

PostgreSQL + Git Adapter + MeaningDelta schema + CLI を実装する。

M2: RDE Record Integration

RDEAssessment / ReviewDecision を MeaningDelta に紐づける。RDEAssessment は JSONB として保持し、後続の schema evolution に耐える構造にする。

M3: Minimal UI Prototype

VSCode extension として、Git diff と MeaningDelta 登録 UI を実装する。

M4: GitHub Integration

Issue / PR / Review Comment と意味履歴を接続する。

M5: AgentRun Integration

ChatGPT app / Claude Code / NotebookLM などの外部出力を記録可能にする。

M6: Team Mode

PostgreSQL のマルチユーザー運用 / Web Console / 権限管理 / 監査ログを実装する。

MX: Multimodal Research Note

画像・音声・音楽・動画への ΔM 拡張を研究ノートとして整理する。

23 v0.1 の明確な非対象

v0.1 では、以下を実装対象外とする。

- Obsidian 全体の再実装
- Cursor 的 AI コードエディタの再実装
- NotebookLM 的資料読解 UI の再実装
- 完全自動 RDE 判定
- 完全自動 merge
- マルチモーダル ΔM 評価
- 組織向け高度権限管理
- 大規模 Graph DB
- 文書生成 AI そのもの

これは縮小ではなく、設計上の節制である。

Kotonoha v0.1 がまず実証すべきことは、ただ一つである。

Git 差分に対して、意味変化 ΔM と RDE 評価を記録できるか。

ここが成立すれば、Kotonoha の基礎は成立する。ここが曖昧なまま外部連携や UI を拡張すると、単なる AI ツール統合基盤に流れてしまう。

24 Multimodal MeaningDelta の位置づけ

画像・音声・音楽・動画への ΔM 評価は、現時点では Kotonoha v0.1 の実装対象ではなく、将来的可能性の方向として提示する。

意味変化 ΔM は本質的にはテキストに限定されない。画像、音声、音楽、動画においても、編集・要約・変換・生成によって意味の保存、変形、補完、逸脱が発生する。

ただし、既存の画像類似度、音響品質、動画一貫性評価をそのまま ΔM 評価と呼ぶべきではない。

それらは下位特徴量であり、RDE が扱うべきなのは、より上位の意味変化である。

- 何が保存されたか
- 何が変換されたか
- 何が補完されたか
- 何が省略されたか
- 何が別の意味へ滑ったか
- その変化は意図されたものか
- 責任・価値・文脈の配置は変わったか

将来的な Multimodal ΔM 評価では、RDE の基本 6 分類を上位判定カテゴリとして維持しつつ、知覚、対象、行為、時間、感情、様式、文化的象徴、制度的文脈、モダリティ間整合性といった補助評価軸を導入する。

Perceptual Axis

知覚的变化。

Object / Entity Axis

対象・人物・物体・主体の変化。

Action / Event Axis

行為・出来事・因果関係の変化。

Temporal Axis

時間順序・継続性・編集構造の変化。

Affective / Tonal Axis

感情・声色・演出トーン・雰囲気の変化。

Stylistic / Genre Axis

様式・ジャンル・表現形式の変化。

Symbolic / Cultural Axis

象徴・文化的文脈・記号的意味の変化。

Contextual / Institutional Axis

報道性・証拠性・責任配置・制度的意味の変化。

Cross-modal Alignment Axis

映像・音声・字幕・音楽・テキスト間の整合性変化。

ただし、この議論は本文中核ではなく、研究ノートまたは将来展望ノートとして保持する。

25 RDE 差異検証メモ

25.1 保存された要素

Kotonoha が意味変化の履歴を管理する基盤であるという中核は保存されている。

25.2 変換された要素

当初の Obsidian / Cursor 中心の作業環境観から、エディタ非依存の Semantic Lineage Core として再定義された。

25.3 補完された要素

Git 連携 DB、Document Object、MeaningState、MeaningDelta、RDEAssessment、ReviewDecision、AgentRun、MCP Gateway、Minimal UI、PostgreSQL 前提が補完された。

25.4 未解決の要素

PostgreSQL スキーマの詳細、RDEAssessment の JSONB 構造、VSCode extension の UI 仕様、GitHub 連携 API、AgentRun ログ形式、権限モデルは未解決である。

25.5 逸脱リスク

Kotonoha が、AI エディタ、Obsidian プラグイン、NotebookLM 代替、Git 拡張のいずれかに矮小化されるリスクがある。

また、Multimodal ΔM や完全自動 RDE 評価など、将来研究段階の内容を実装済み能力のように見せてしまうリスクがある。

25.6 次回更新方針

今回は、Kotonoha Core v0.1 の PostgreSQL スキーマ、CLI コマンド体系、VSCode Minimal UI 仕様を分離して具体化する。

26 要約

Kotonoha は、Git などの内容履歴を物理的アンカーとして用いながら、文書・議論・設計・実装における意味状態と意味変化 ΔM を記録し、RDE によって監査可能にする Semantic Lineage System である。

Obsidian、Cursor、VSCode、ChatGPT app、Claude Code、NotebookLM は、Kotonoha の本体ではない。それらはすべて、Kotonoha Core に接続されるクライアントまたは外部エージェントである。

Kotonoha v0.1 がまず実証すべきことは、Git 差分に対して、MeaningDelta と RDEAssessment を PostgreSQL 上に記録できるかである。

この最小 Core が成立すれば、Kotonoha は単なる文書管理でも、AI 編集支援でもなく、生成 AI 時代における意味の来歴管理基盤として成立する。

付録 A Obsidian 用メタデータ

```
---
title: "Kotonoha 構想：意味履歴を管理する Semantic Lineage System"
created: 2026-05-17T00:00:00+09:00
updated: 2026-05-17T00:00:00+09:00
author: "Tomoyuki Kano <tomyuk@zyxcorp.jp>"
tags: [Kotonoha, SLS, RDE, MeaningDelta, SemanticLineage, Git, PostgreSQL, AI 設計]
source_type: chatgpt
source_id: "kotonoha-semantic-lineage-concept"
document_type: concept_note
status: draft
links:
  - [[RDE]]
  - [[MeaningDelta]]
  - [[Semantic Lineage System]]
  - [[Kotonoha Minimal UI]]
---
```